

Day 1 : 간이 로그인 검증 프로그램 & Gradle

4반 이름: 박인우

Chapter 1. 코드 이해

-먼저 프로그램의 기본 뼈대를 생성한다. 패키지 안에 App클래스, JVM이 main() 실행하는 형태

```
package login_server;

public class App {

    public static void main(String[] args) {

    }

}
```

- 프로그램에서 사용할 도구들을 가져온다.

```
// 현재 Java 프로그램에서 사용하고 하는 외부 라이브러리를 명시한다.
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
```

-회원 정보를 저장할 Map 추가, 이 안에다 아이디, 비밀번호를 넣는다.(App클래스 안, main바깥)

```
// Map을 생성한다.
// Map : 이름으로 값을 관리하는 데이터 관리 요소
// 가입 되어 있는 회원들의 아이디와 비밀번호를 저장해둘 저장소로
// 사용할 것이다.

private static final Map<String, String> db = new HashMap<>();
```

-static 블록으로 초기 회원 데이터 추가

```
// static 코드블럭
// 원래 클래스안에 있는 모든 요소들은 객체라는 것을 만들어야
// 사용할 수 있다.
// static이 붙어 있는 것들은 바로 사용이 가능하다.
// static 코드 블럭은 프로그램 실행시 자동으로 동작되는 부분을
// 만들고 싶을 때 사용한다.
static {
    // Map에 데이터를 저장한다.
    // 데이터를 관리할 때 사용할 이름을 사용자 ID 로 사용하고
    // 관리할 값을 비밀번호로 사용 하겠습니다.
    db.put("alice", "pass1234");
    db.put("bob", "mySecret");
}
```

- 키보드 입력을 위한 객체를 생성한다.(main() 안)

```
// 키보드 입력을 위한 객체를 생성한다.
```

```
// 키보드를 통해 라인단위로 입력을 받는다.
Scanner sc = new Scanner(System.in);

System.out.println("=== Day 1: 간이 로그인 시스템 ===");
```

-아이디와 비밀번호 입력

```
// 사용자의 로그인 정보를 입력받는다.
System.out.print("아이디: ");
// nextLine() : 키보드로 한 줄 입력 받는다.
// trim() : 좌우 공백 제거
String id = sc.nextLine().trim();
```

```
System.out.print("비밀번호: ");
```

```
String pw = sc.nextLine().trim();
```

-아이디 존재 여부 검사, `db.containsKey(id)`가 입력받은 아이디가 Map의 key로 존재하는지 검사 존재하지 않는 아이디라면, `return`으로 `main()`을 끝냄

```
// Map에 사용자가 입력한 ID로 저장되어 있는 값이 있는지 확인
// if 문 : ( ) 의 코드의 결과가 참인 경우에만 수행되는 코드 관리 요소
// Map에 입력한 ID 저장된 값이 없으면 if 문 내부의 코드가
// 동작한다.
if (!db.containsKey(id)) {
    // 출력
    System.out.println("존재하지 않는 ID입니다.");
    // main 종료 -> 프로그램 종료
    return;
}
```

-아이디가 있으면 비밀번호 비교

```
// 평문비교 (보안 취약점)

// db.get(id) : 맵에서 id 변수에 들어있는 문자열을 이름으로 하는
// 값을 가지고 온다.
// .equals(pw) : pw와 같은지 확인한다.
if (db.get(id).equals(pw)) {
    System.out.println("로그인 성공! 환영합니다");
} else {
    System.out.println("비밀번호가 틀렸습니다");
}
```

간단한 정리

회원정보가 들어갈 **Map** 생성 -> **static** 블록에서 아이디 비밀번호 저장 -> **main()** 실행 -> 키보드 객체를 이용해서 아이디와 비밀번호 입력 -> 아이디 존재여부 확인해서 저장된 비밀번호와 입력값 비교 -> 로그인 결과 출력

Chapter 2. Gradle 이용하여 jar를 만들고 실행

1. 프로젝트 최상위 폴더에서 터미널로 `./gradlew clean build` 실행
 2. `app > build > libs` 에 `app.jar` 파일이 생성되어 있는지 확인한다.
 3. 터미널에서 `java -cp app/build/libs/app.jar login_server.App` 실행
-

Chapter 3. 현재 코드의 문제점

보안 문제

비밀번호를 평문으로 저장해 코드가 노출되면 비밀번호도 함께 노출된다.

데이터 관리 문제

Map 데이터는 메모리에 있어 프로그램을 종료하면 사라진다.

여러 프로그램들이 같은 회원 데이터를 공유하기 어렵다

부록. 궁금했던 점

scanner라는 객체를 사용하는 이유

System.in으로 들어오는 원시 입력 데이터를 문자열·정수·실수처럼 사용하기 쉬운 **Java** 값으로 읽고 변환하기 위해서

System.in을 직접 사용하면 입력이 문자나 바이트 단위로 들어와서 처리하기 불편

```
int data = System.in.read();
```

사용자가 **A**를 입력해도 결과는 문자 **"A"**가 아니라 숫자 코드인 **65**로 들어올 수 있다. 여러 글자를 한 문장으로 만들려면 직접 반복해서 읽고 조합해야함.

반면 **Scanner**를 사용하면:

```
String id = sc.nextLine();
```

```
int age = sc.nextInt();
```

```
double height = sc.nextDouble();
```

처럼 원하는 형태로 바로 읽을 수 있다.

`nextLine()` → 한 줄을 `String`으로 읽기

`next()` → 공백 전까지 한 단어 읽기

`nextInt()` → 정수로 읽기

`nextDouble()` → 실수로 읽기

혹시 스캐너에 내가 따로 임의로 추가 메서드를 부여해서 사용도 가능한가?

`Scanner`에 직접 메서드 추가 → 불가능

App에 `Scanner`를 사용하는 메서드 작성 → 가능

`Scanner`를 포함한 새로운 `InputReader` 클래스 작성 → 가능하며 구조적으로 더 좋음

`Scanner` 자체를 변경하는 것이 아니라, `Scanner`를 내부에서 활용하는 새로운 클래스나 메서드를 만들어 원하는 기능을 추가하는 방식으로 사용한다.

결론 : 역할별 클래스를 각 파일에 만들고, `main()`은 필요한 객체를 생성해서 메서드를 호출하며 전체 흐름을 연결한다.

Static 있고 없고 차이

`static` 없음

- 변수와 메서드가 객체에 소속
- 객체를 먼저 만들어야 사용 가능
- 객체마다 값이 따로 존재
- 새 객체는 다시 초기값부터 시작!

static 사용

- 변수와 메서드가 클래스에 소속
- 객체를 만들지 않아도 사용 가능
- 클래스 전체에 하나만 존재
- 모든 객체가 같은 값을 공유!
- 일반적으로 JVM 종료까지 유지